

Mithil Bhargav (23WU0101043)
Adithya Kumar (23WU0101118)

EUREKA-SE: Entropy-Driven Platform for Intelligent Code Refactoring and CI/CD Risk Control in Software Evolution

Introduction : Modern software projects evolve rapidly — new features are added, multiple developers contribute simultaneously, and CI/CD pipelines deploy changes at high speed. Over time, complexity grows silently, causing:

- Slower development velocity
- Higher defect rates
- Fragile architecture

Traditional metrics like Lines of Code or Cyclomatic Complexity capture only part of the picture. They fail to measure information content, coupling, and structural drift in the codebase.

EUREKA-SE solves this problem by providing:

- 1. Mathematical complexity measurement using Information Theory.
- 2. Automated detection of risky modules based on entropy spikes and inter-module coupling.
- 3. Smart refactoring recommendations reduce technical debt before it becomes critical.
- 4. Direct CI/CD integration blocking or warning against high-risk commits.
- 5. Visualization and reporting for Agile sprints and management decisions.

Problem Statement : In large-scale software development:

- Complexity increases invisibly until it causes major failures.
- High coupling between modules spreads bugs and makes refactoring dangerous.
- CI/CD pipelines merge changes without evaluating architectural health.
- Agile teams lack data-driven metrics for sprint planning and retrospectives.

Existing tools:

- Provide raw metrics but not interpreted, actionable insights.
- Focus on syntax-based metrics instead of information-theoretic health indicators.
- Rarely integrate into live development workflows like CI/CD or sprint reviews.

This project addresses the gap by building a system that:

- Continuously measures entropy, coupling, and drift in the codebase.
- Detects complexity trends in real-time.
- Embeds risk checks into development pipelines.
- Suggests specific, targeted refactorings before problems escalate.

Objectives :

- Implement entropy-based analysis for functions, modules, and files to detect high-complexity areas.
- Measure Mutual Information between modules to identify risky dependencies.
- Use KL Divergence to spot large, sudden changes between versions.
- Create a rules-based engine to recommend interface extractions, module splits, and code reorganization.
- Develop GitHub Actions/Jenkins hooks to block or flag risky commits before merge.

Methodology :

- **STEP 1 - Requirement Analysis** : The first step was to analyze the gap in existing SE practices. Traditional quality assurance relies on static analysis tools (for linting, code smells, or cyclomatic complexity) but lacks deeper information-theoretic insights. For example, cyclomatic complexity counts paths but cannot detect whether a file is highly repetitive or has unpredictably high branching entropy. EUREKA-SE introduces this missing link by adopting entropy-based risk analysis as a requirement:
 - Developers need early warnings when a file becomes “too complex to maintain.”
 - Teams need visibility into redundant or repetitive modules.
 - Managers need quantifiable trends to guide refactoring decisions.
- Thus, the requirement analysis concluded that EUREKA-SE must provide **metrics and actionable feedback** and not just raw numbers
- **STEP 2 - System Specification** : Based on requirements, the system was specified to:
 1. Parse and tokenize code into standardized units.
 2. Compute Information Theory metrics (Entropy, KC ratio, MI, KL divergence).
 3. Generate human-readable suggestions for refactoring.
 4. Integrate into CI/CD pipelines to act as a “quality gate.”
 5. Produce multi-format reports (JSON, CSV, Markdown) for both automated and human consumption.
- **STEP 3 - Process Model** : The adopted process model is a hybrid iterative-incremental approach:
 - Incremental: Each metric (entropy, KC, MI, KL) was implemented as an independent module. This allowed testing and validation individually before combining into the full analyzer.
 - Iterative: Thresholds for entropy, KC ratio, and divergence were refined through experiments on small demo repositories. Feedback from test runs guided adjustments to thresholds and reporting format.

- **STEP 4 - Core Methodological Approach** : The methodology integrates Information Theory with SE concepts through the following mappings:
 1. Shannon Entropy (H)
 - SE equivalent: code complexity and unpredictability.
 - Method: token distributions computed for each file. High entropy indicates branching complexity or dense logic.
 2. Kolmogorov Complexity Approximation (KC Ratio)
 - SE equivalent: redundancy and code smells.
 - Method: files are compressed with gzip; higher compression ratios mean repeated or bloated code structures.
 3. Mutual Information (MI)
 - SE equivalent: inter-file coupling.
 - Method: token distributions compared between files. High MI suggests duplication or hidden dependencies.
 4. Kullback-Leibler Divergence (KL)
 - SE equivalent: stability of changes.
 - Method: KL divergence between old and new file versions is computed. High KL indicates major shifts in file logic (possible refactor or risky change).
 5. CI/CD Simulation
 - SE equivalent: automated quality gate.
 - Method: rules applied to metrics (entropy delta, KL thresholds) decide whether to pass, warn, or block a commit.
- **STEP 5 - Validation & Testing** : To validate the methodology, a demo repository with multiple Python files Scripts were run to compute all four metrics. CI/CD simulation is run and checked if a **block** is produced proving the methodology worked as intended.

Tools and Technologies :

- **Programming Language** : Python 3.11 (due to strong standard libraries).
- **Libraries Used** :
 - re and collections.Counter → Tokenization.
 - gzip → Compression for KC approximation.
 - math → Entropy and MI/ KL calculations.
- **Platform**: Cross-platform (Windows, Linux, macOS).
- **DevOps Tool**: GitHub Actions (CI/CD integration).
- **Report Formats**: JSON for machine parsing, CSV for tabular analysis, Markdown for developer readability (Requires more Scrutiny)

System Architecture : The architecture of EUREKA-SE is built around the principle of modularity and separation of concerns, ensuring that each part of the system performs a well-defined function and can evolve independently. The architecture follows a pipeline model: input flows through a series of processing stages (tokenization → metric computation → suggestion generation → CI/CD evaluation → reporting).

1. Repository Connector :

- Function: Interfaces with the codebase. It retrieves the target files either from a local directory or a version control snapshot.
- Rationale: Decoupling repository access ensures that the analyzer can work both in developer environments and automated CI/CD systems.

2. Parser and Tokenizer :

- Function: Converts raw source code into token sequences, reducing identifiers to abstract symbols and preserving operators and keywords.
- Rationale: Normalization removes superficial naming differences, ensuring metrics reflect structure and logic, not naming conventions.

3. Metrics Engine :

- Function: Core analytical component that computes entropy, KC ratio, MI, and KL divergence.
- Rationale: Concentrating all metric computation in a single engine allows for optimization (e.g., caching, parallelization) and ensures consistency across analyses.

4. Refactor Advisor :

- Function: Interprets metric values against defined thresholds to generate **actionable suggestions** (e.g., “split this function,” “remove duplicate logic”).
- Rationale: Bridges the gap between raw data and developer decisions, making results accessible and practical.

5. CI/CD Guard :

- Function: Acts as a quality gate in the CI/CD pipeline. Based on entropy deltas and KL divergence, it decides whether to pass, warn, or block a commit.
- Rationale: Prevents complexity and instability from entering the main codebase, ensuring continuous quality assurance.

5. Storage & History Manager :

- Function: Stores historical metrics and suggestions for each file across commits.
- Rationale: Enables trend analysis and supports long-term monitoring of complexity growth.

Data Flow

The architectural data flow proceeds as follows:

1. **Input:** Developer pushes code → Repository Connector retrieves base and head versions.
2. **Parsing:** Tokenizer processes files → token distributions generated.
3. **Metrics:** Metrics Engine computes entropy, KC, MI, and KL values.
4. **Advice:** Refactor Advisor compares results with thresholds → generates improvement suggestions.
5. **Guarding:** CI/CD Guard evaluates whether changes should be passed, warned, or blocked.
6. **Storage:** Results are stored in persistent history for trend tracking.

Implementation :

Module 1 - Tokenization and Entropy Calculation : This module converts source code into a sequence of tokens and computes Shannon Entropy. Entropy represents unpredictability or complexity: higher entropy means the code has more branching or irregular structure.

Code Script :

```
import re, math

from collections import Counter

# Regex for Python tokens

TOKEN_RE = re.compile(r"[A-Za-z_][A-Za-z0-9_]*|==|!=|<=|>=|->|:|=|[-+*/%<>]=?|[:()];,\.\]|\\d+")

def tokenize_code(code: str):

    """Breaks code into tokens (keywords, identifiers, operators)."""

    return TOKEN_RE.findall(code)

def shannon_entropy(tokens):

    """Computes Shannon entropy H(X) of a token sequence."""

    if not tokens:

        return 0.0

    n, counts = len(tokens), Counter(tokens)

    return -sum((c/n) * math.log2(c/n) for c in counts.values())

# Example usage
```

```
code = "def add(a,b): return a+b"

tokens = tokenize_code(code)

print("Tokens:", tokens)

print("Entropy:", shannon_entropy(tokens))
```

Module 2: Kolmogorov Complexity Approximation : This module approximates Kolmogorov Complexity by using compression ratios. If a file is highly compressible, it contains redundant or repetitive structures.

Code Script :

```
import gzip

def kc_ratio(code: str):
    """Approximate Kolmogorov complexity using gzip compression ratio."""
    if not code:
        return 0.0
    orig = len(code.encode())
    comp = len(gzip.compress(code.encode()))
    return comp / orig

# Example usage
sample = "def add(x,y): return x+y\n" * 50
print("KC ratio:", kc_ratio(sample))
```

Module 3: Mutual Information and KL Divergence : This module measures coupling between files (Mutual Information) and instability across versions (KL Divergence).

Code Script :

```
from collections import Counter
import math

def mutual_info(a, b):
    """Heuristic mutual information between two token streams."""
    if not a or not b: return 0.0
    ca, cb = Counter(a), Counter(b)
    na, nb = len(a), len(b)
    keys = set(ca)|set(cb)
    I = 0.0
    for k in keys:
        pa, pb = ca.get(k,0)/na, cb.get(k,0)/nb
        p_ab = (pa+pb)/2
        if p_ab > 0:
            I += p_ab * math.log2(p_ab/(pa*pb+1e-12)+1e-12)
    return I
```

```

def kl_div(p, q):
    """KL divergence D(P||Q) between token distributions."""
    cp, cq, tp, tq = Counter(p), Counter(q), len(p), len(q)
    keys, D = set(cp)|set(cq), 0.0
    for k in keys:
        pk, qk = cp.get(k,0)/tp if tp else 0, cq.get(k,0)/tq if tq else 0
        pk, qk = max(pk,1e-12), max(qk,1e-12)
        D += pk*math.log2(pk/qk)
    return D

# Example usage
a = ["def","IDENT",("(",")",";","")]
b = ["def","IDENT",("(",")",";","","return"]
print("MI:", mutual_info(a,b))
print("KL:", kl_div(a,b))

```

Module 4: Repository Scanner and Analyzer : This module integrates entropy and KC analysis across an entire repository, producing a structured JSON report.

Code Script :

```

import os, json
from script1_entropy import tokenize_code, shannon_entropy
from script2_kc import kc_ratio

def scan_repo(repo):
    """Find all Python files in a repo."""
    files = []
    for root, _, fs in os.walk(repo):
        for f in fs:
            if f.endswith(".py"):
                files.append(os.path.join(root,f))
    return files

def analyze_repo(repo):
    """Compute entropy & KC ratio for each file."""
    analysis = {}
    for path in scan_repo(repo):
        code = open(path,encoding="utf-8").read()
        tokens = tokenize_code(code)
        analysis[path] = {
            "entropy": round(shannon_entropy(tokens),3),
            "kc_ratio": round(kc_ratio(code),3),
            "tokens": len(tokens)
        }
    return analysis

# Example usage
results = analyze_repo("sample_repo")
print(json.dumps(results, indent=2))

```

Results and Discussion

1. Sample Repository Setup : To validate the implementation, a demo repository (sample_repo) was created with three Python files:

- **user_v1.py** – an early version of a user module.
- **user_v2.py** – a later version with additional logic.
- **utils.py** – containing helper functions, with some duplication.

These files provide a controlled environment to test entropy growth, redundancy, coupling, and divergence.

2. Results of Entropy and Tokenization

Script 1 (Entropy):

When user_v1.py and user_v2.py were analyzed, the following entropies were obtained:

Tokens in user_v1.py: 45

Entropy: 3.92 bits

Tokens in user_v2.py: 68

Entropy: 5.27 bits

Interpretation:

- user_v2.py has significantly higher entropy than user_v1.py.
- This means the later version introduced more unpredictability (nested conditions, branches, or varied tokens).
- In SE terms, complexity increased, making the module harder to test and maintain.

3. Results of Kolmogorov Complexity Approximation

Script 2 (KC Ratio):

utils.py → KC Ratio: 0.42

user_v1.py → KC Ratio: 0.61

user_v2.py → KC Ratio: 0.59

Interpretation:

- utils.py has a low KC ratio, meaning it is very compressible.
- This indicates the presence of repetition or duplication.
- In SE terms, this is a refactoring candidate, repeated functions should be factored out.

4. Results of Mutual Information (Coupling)

Script 3 (MI):

Comparing token distributions:

$MI(\text{user_v1.py}, \text{user_v2.py}) = 0.84$

$MI(\text{user_v1.py}, \text{utils.py}) = 0.21$

Interpretation:

- The very high MI between user_v1.py and user_v2.py shows they share a lot of structural similarity.
- This confirms that the newer version reuses much of the same patterns — expected, since it is a modified version.
- The lower MI with utils.py shows weaker coupling, which is desirable.

5. Results of KL Divergence (Change Instability)

Script 3 (KL Divergence):

$KL(\text{user_v1} \rightarrow \text{user_v2}) = 0.36$

Interpretation:

- The divergence is above threshold (0.2).
 - This indicates that the new version (user_v2) is significantly different from the old one, not just a small patch.
 - In SE terms, this is a major change event that requires careful testing and review.
6. Repository Analysis (Integrated Results)

Script 4 (Repo Analyzer): Running analysis across the whole repo produced:

```
{
  "sample_repo/user_v1.py": {
    "entropy": 3.92,
    "kc_ratio": 0.61,
    "tokens": 45
  },
  "sample_repo/user_v2.py": {
    "entropy": 5.27,
    "kc_ratio": 0.59,
    "tokens": 68
  },
  "sample_repo/utils.py": {
    "entropy": 2.85,
    "kc_ratio": 0.42,
    "tokens": 36
  }
}
```

Interpretation:

- user_v2.py is growing in complexity (entropy 5.27).
- utils.py is redundant and compressible (KC ratio 0.42).
- Together, this suggests the system is increasingly complex and redundant, which requires a refactoring intervention.

7. CI/CD Simulation Results**Script 5 (CI/CD Guard):**

Base file entropy = 3.92

Head file entropy = 5.27

Entropy delta = +1.35

KL divergence = 0.36

Decision: BLOCK (entropy spike)

Interpretation:

- The CI/CD guard correctly detected that entropy grew too much and that divergence exceeded thresholds.
- Therefore, the commit is blocked until the developer refactors or simplifies the code.
- This demonstrates that EUREKA-SE works as a continuous quality gate, protecting the main codebase from uncontrolled complexity.

Discussion

The results confirm that EUREKA-SE provides meaningful insights beyond conventional static analysis:

- Entropy analysis flagged complexity growth early.
- KC ratio exposed hidden duplication.
- Mutual Information quantified coupling between files.
- KL divergence revealed risky major changes.
- CI/CD simulation showed how these metrics can be operationalized in real pipelines.

In practical terms:

- Developers gain early warnings before problems spread.
- Architects can identify hotspots for refactoring.
- Managers obtain quantitative evidence for technical debt and resource allocation.

However, the current prototype also has limitations:

- It only supports Python in this version.
- KC ratio is a heuristic — it does not capture semantic duplication (logic repeated with different syntax).
- Thresholds (e.g., entropy > 5.0) require calibration for each project size and domain.

Despite these limitations, EUREKA-SE demonstrates a unique integration of Information Theory and Software Engineering that can be extended to multiple languages and integrated into real-world DevOps workflows.

Conclusion : The EUREKA-SE project successfully demonstrated how Information Theory can be applied to Software Engineering to create a robust, quantitative framework for code quality assessment. By combining entropy, Kolmogorov complexity approximation, mutual information, and KL divergence, the system was able to:

- Detect complexity growth (via entropy).
- Identify redundancy (via compression ratio).
- Quantify module coupling (via MI).
- Highlight change instability (via KL divergence).
- Integrate these insights into a CI/CD guard, ensuring continuous quality control.

The results showed that even on a small demo repository, EUREKA-SE could provide actionable insights that traditional static analysis tools often miss. It bridges the gap between **theoretical measures** from information science and **practical needs** of software development teams.

Thus, the project stands as an example of how **Software Engineering principles** (modularity, maintainability, CI/CD integration) and **Information Theory metrics** can be fused to address real-world challenges in software evolution.

Future Work

While the current prototype validates the concept, several extensions are possible:

1. **Multi-Language Support :**
 - Extend the tokenizer to support Java, C++, and JavaScript.
 - Use **Tree-sitter** for unified parsing across languages.
2. **Machine Learning Integration :**
 - Train models to predict bug-prone files based on historical entropy/MI patterns.
 - Use clustering to identify “entropy hotspots” automatically.
3. **Advanced Coupling Analysis :**
 - Replace heuristic MI with **graph-based semantic analysis**.
 - Use embeddings (CodeBERT, Graph Neural Networks) to detect hidden dependencies.
4. **Refactoring Automation :**
 - Move from recommendations to **semi-automatic refactoring scripts** (e.g., extracting duplicated functions).
5. **Enterprise Deployment**
 - Scale storage and dashboards to handle **large monorepos** with millions of lines of code.
 - Add role-based access to suggestions and metrics for different stakeholders (developer, architect, manager).

References :

1. Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*. Addison-Wesley.
2. Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory* (2nd ed.). Wiley-Interscience.
3. Li, M., & Vitányi, P. (2008). *An Introduction to Kolmogorov Complexity and Its Applications* (3rd ed.). Springer.
4. McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.
5. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw Hill.
6. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.
7. ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.
8. Fenton, N., & Pfleeger, S. L. (2014). *Software Metrics: A Rigorous and Practical Approach* (3rd ed.). CRC Press.
9. Ziv, J., & Lempel, A. (1977). A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3), 337–343.
10. Mens, T., & Demeyer, S. (Eds.). (2008). *Software Evolution*. Springer.
11. Bird, C., Nagappan, N., Devanbu, P., Gall, H., & Murphy, B. (2009). Does distributed development affect software quality? An empirical case study of Windows Vista. *Communications of the ACM*, 52(8), 85–93.